Шпаргалка и Полный Разбор Кода к Защите

Системное программирование .NET 8.0 — Станция испытания аккумуляторных ячеек

РАЗДЕЛ 1. ГЛАВНАЯ ЛОГИКА И МНОГОПОТОЧНОСТЬ (PROGRAM.CS)

В1: Почему здесь используется ThreadPool.QueueUserWorkItem, а не new Thread или async/await?

Согласно требованиям ТЗ (пункт 4), **запрещено создавать потоки вручную через** `new Thread`, так как это создает огромную нагрузку на ОС при выделении ресурсов и переключении контекста. Вместо этого используется системный пул потоков .NET (`ThreadPool`), который эффективно управляет готовым набором потоков. Каждая ячейка партии ставится в очередь как отдельный рабочий элемент через `ThreadPool.QueueUserWorkItem`, а для ожидания завершения всех задач применяется `CountdownEvent`.

В2: Зачем здесь SemaphoreSlim и почему не обычный lock?

Станция имеет ограниченное количество физических слотов зарядного шкафа (по умолчанию 4). `SemaphoreSlim` идеально подходит для ограничения доступа к ресурсу с заданным числом одновременных мест. Перед началом зарядки поток вызывает `semaphore.Wait(cancellationToken)`. Если свободных слотов нет, поток ждет, но это **ожидание прерываемое** — если прилетит отмена, поток сразу выйдет. Вход и выход обернуты в конструкцию `try / finally`, чтобы гарантировать освобождение семафора даже при исключениях.

В3: Почему для калибровки используется именно Mutex, а не lock или SemaphoreSlim?

В ТЗ (пункт 6) строго указано: калибровочный стенд единственный в лаборатории и обрабатывает ровно одну ячейку за раз, причем **запрещено использовать** `lock`. `Mutex` — это примитив синхронизации ядра ОС. Для ожидания мьютекса одновременно с токеном отмены используется метод `WaitHandle.WaitAny`. Это позволяет потоку не зависнуть в очереди намертво, а корректно отмениться, если сработает `CancellationToken`.

В4: Как реализована аварийная остановка и что происходит с ресурсами при отмене?

Создан глобальный `CancellationTokenSource`, который слушает нажатие комбинации `Ctrl+C` в консоли (через `Console.CancelKeyPress`) или таймер автоотмены. Один и тот же токен передается во все рабочие потоки. Внутри тяжелого цикла диагностики периодически вызывается `token.ThrowIfCancellationRequested()`. При отмене все текущие операции прерываются, ячейки получают статус `Canceled`, а блоки `finally` гарантируют, что все занятые `SemaphoreSlim` и `Mutex` мгновенно освобождаются, предотвращая утечки и зависания.

В5: Как вы избегаете состояний гонки (race conditions) при обновлении статистики и выводе в консоль?

Для вывода логов используется синхронизация через ключевое слово `lock (_lockObj)` в классе `StationLogger`, чтобы строчки в консоли не перемешивались. Счетчики успешных и отмененных ячеек инкрементируются атомарно через `Interlocked.Increment`, а для сохранения результатов используется потокобезопасная коллекция `ConcurrentDictionary`.

РАЗДЕЛ 2. ПОТОКОБЕЗОПАСНЫЙ ЖУРНАЛ (STATIONLOGGER.CS)

В6: Зачем выносить логирование в отдельный класс?

Это принцип разделения ответственности (Single Responsibility). Класс `StationLogger` инкапсулирует всю логику вывода в консоль, форматирования временных меток и потокобезопасности, чтобы основной код программы (`Program.cs`) не загромождался техническими деталями печати.

В7: Зачем здесь нужен lock (_lockObj)? Что будет, если его убрать?

Множество потоков из `ThreadPool` работают параллельно и одновременно вызывают методы логирования. Без блокировки `lock` потоки будут одновременно писать в `Console.WriteLine`, из-за чего строки в консоли перемешаются, разорвутся пополам или "склеятся" (состояние гонки / race condition). Конструкция `lock` гарантирует, что в критическую секцию вывода одновременно заходит только один поток.

В8: Почему объект блокировки — это private readonly object _lockObj = new();, а не this?

Это правило хорошего тона при работе с многопоточностью. Если блокироваться по `this` (экземпляру класса) или по типу через `typeof(StationLogger)`, внешний код теоретически сможет получить доступ к этому же объекту блокировки и случайно вызвать взаимную блокировку (**deadlock**). Создание приватного `readonly` объекта гарантирует, что заблокировать этот монитор извне невозможно.

В9: Что делают форматирования [Thr-{threadId,2}] и {status,-22}?

Это интерполяция строк с выравниванием:
• `{threadId,2}` — выравнивает ID потока по ширине в 2 символа, чтобы таблица логов не "плясала" при смене однозначных ID на двузначные.
• `{status,-22}` — выравнивает строку статуса по левому краю с фиксированной шириной в 22 символа, чтобы разделитель `|` находился на одной линии.

РАЗДЕЛ 3. МОДЕЛИ ДАННЫХ (MODELS: CELL.CS, CELLSTATUS.CS, PROCESSINGRESULT.CS)

В10: Зачем нужны отдельные файлы Cell.cs, CellStatus.cs и ProcessingResult.cs?

• `CellStatus.cs` — перечисление (enum) всех 9 этапов жизненного цикла ячейки по ТЗ (от `Pending` до `Completed`, `Canceled` и `Faulted`).
• `Cell.cs` — модель ячейки. Поля `Id` и длительности имеют инициализатор `init` (задаются один раз при создании партии и не меняются во избежание состояния гонки), а `Status` имеет `set` для изменения потоками.
• `ProcessingResult.cs` — класс слепка итоговых результатов каждой ячейки для сохранения в `ConcurrentDictionary` и последующего формирования финального отчета.

💡 **Совет перед показом преподавателю:**

Пробегись глазами по этим пунктам, запусти проект в Visual Studio (нажми F5). Если препод попросит сымитировать отмену — нажми **Ctrl + C** прямо во время работы консоли или выстави в коде `AutoCancelTimeoutMs = 2000`. Все ресурсы гарантированно освобождаются!